

NBchat — 爆款差距分析与最终汇报指南

经过全网搜索对比 Arattai、XChat、Signal、Telegram、Volla Messages 等产品的核心竞争力和 2025 年 Chat UX 最佳实践，以下是系统的差距分析和汇报策略。

一、我们 VS 爆款产品：差距地图

● 产品级差距（决定产品能否"出圈"）

维度	爆款标准	我们	差距
首次体验	Welcome Bot 引导 → 5 秒内完成首个关键操作	空白状态，需手动找 ID 加好友	●
通知系统	"Sarah 在 #产品组 @了你" → 14% CTR	无推送通知	●
离线消息	不在线也能收消息（FCM/APNs）	WebSocket 断连就收不到	●
消息撤回持久化	撤回后显示原消息已被截屏警告	撤回即消失，无防截屏	●
群聊 @所有人	一键通知全体，支持免打扰豁免	只能手动 @每个人	●
AI 主动服务	汇总未读、总结讨论、推荐回复	被动响应 @千问小助手	●
富媒体贴纸/GIF	内置 sticker picker + GIPHY	只有 emoji 文字	●
语音通话质量	TURN 中继 + 降噪 + 回声消除	无 TURN 服务器部署	●
桌面端	Electron/Tauri 原生体验	纯 Web，无离线能力	●
支付/生态集成	红包、转账、小程序	无	●

● UX 细节差距（决定用户是否"上瘾"）

细节	最佳实践	我们
优先收件箱	"3 条重要，47 条普通"	仅统一未读计数
新消息分割线	"↓ 34 条新消息" 分隔符	无
流式 AI 回复	逐 token 渲染，可中途取消	1.8s 后整段出现
智能免打扰	你在群里活跃时自动抑制通知	需手动设置免打扰
快捷投票	"午饭吃啥？🍕🍲🥗（10 分钟）"	无
草稿保存	未发送的消息自动保存	切换会话即丢失
最近表情	记住你常用的 8 个 emoji	每次都是默认 16 个
截屏通知	阅后即焚消息被截屏时通知发送者	无

二、可以补上的高价值 Idea（投入 1~3h 每个）

🔔 AI 对话总结 (1.5h) — "今天聊了什么"

进入群聊时，如果未读消息 > 20 条，AI 自动生成一段摘要：

"你错过的：Alice 和 Bob 讨论了周末聚餐的地点和时间，最终决定周六中午 12 点在西门口集合。
Charlie 分享了作业的参考链接。"

技术：把最近未读消息喂给 DeepSeek API，输出格式为 3 行以内中文摘要。在消息列表顶部显示一个可折叠的摘要卡片。

🔔 欢迎 Bot (1h) — "让新用户 5 秒内上瘾"

新注册用户登录后，不是看到空白页面，而是收到一条来自系统 Bot 的消息：

"👋 欢迎来到 NBchat! 这是一个 Jakarta EE 全栈聊天系统。你可以试试：

1. 右侧搜索 **alice** 或 **bob** 加好友
2. 在群聊里 @千问小助手 问任何问题
3. 点击 + 创建你的第一个群聊 赶紧试一下吧! 🚀"

技术：注册时 `AuthDao.createUser()` 调用 `ChatDao.saveSystemMessage()` 给自己发一条欢迎消息。前端 `app.js boot()` 时自动选中该会话。

🔔 快捷投票 (1h) — "5 秒发起一个投票"

输入 `!poll 午饭吃啥?` 🍕 🍝 🥗 → 系统自动渲染一个带选项按钮的投票卡片，群成员点击投票，10 分钟后自动结束并显示结果。

技术：前端检测 `!poll` 前缀 → 解析问题和选项 → 渲染投票卡片 → 每个选项是 `INSERT IGNORE` 入 `message_reactions` 表（复用已有表结构）→ 10 分钟倒计时后锁定。

🔔 "跳转到最新"浮标 (15min) — "不用滚到手酸"

当用户向上翻阅历史时，右下角浮现一个 ↓ `回到最新` 按钮，点击后平滑滚动到底部。

技术：监听 `messageList` 的 `scroll` 事件，距底部 > 300px 时显示浮标。

🔔 草稿保存 (30min) — "切换会话不丢内容"

每个会话在 `localStorage` 中存一份 `composer_draft_{conversationId}`。切换会话时保存当前输入框内容，切回来时恢复。

技术：`selectConversation()` 中加 `localStorage.setItem('draft_' + oldId, input.value)` 和 `input.value = localStorage.getItem('draft_' + newId) || ''`。

三、汇报策略：让评委记住你

3.1 开场 30 秒的黄金钩子

"各位老师好，我做的是一个全功能的实时聊天系统 NBchat。它不只是发消息——它支持群聊管理、朋友圈、语音通话、AI 智能助手、阅后即焚、消息反应和数据可视化。整个系统从数据库到前端全部由我

一个人完成，15,000+ 行代码，现在已经部署在腾讯云上，任何人都可以通过域名访问。"

钩子三要素：功能广度 + 规模量化 + 可访问证明。

3.2 技术栈汇报技巧

不要念技术列表。讲故事：

"我选择 Jakarta EE 作为后端框架，因为它和 Spring Boot 一样是 Java 企业级标准，但更轻量。数据库我用 JDBC 手写 SQL——没有用 ORM，这样我能完全控制查询性能。前端我刻意不用 React 或 Vue，全原生 JavaScript，这样包体积只有 50KB，首屏加载不到 1 秒。"

核心信息：你做的每一个技术选择都有理由，不是"因为老师教的"。

3.3 核心模块汇报顺序

- 1. 认证系统（1 分钟）— BCrypt + 频率限制 + QQ 邮箱 + 前后端双校验
- 2. 实时通信（2 分钟）— WebSocket 事件路由 + 语音 WebRTC + 三级降级
- 3. 聊天功能（2 分钟）— 消息反应/撤回/引用/@补全/阅后即焚/emoji
- 4. AI 助手（1 分钟）— DeepSeek API + 提示词工程 + 上下文管理
- 5. 动效系统（1 分钟）— Canvas 粒子物理/弹簧入场/刮刮乐/爱心粒子/玻璃质感
- 6. 管理后台（1 分钟）— 仪表盘/用户管理/操作日志/群组解散

3.4 评委必问的 5 个问题（提前准备）

问题	你的回答要点
"为什么用 Jakarta EE 而不是 Spring Boot? "	Jakarta EE 是 Java 官方标准，Jersey REST + HK2 DI 比 Spring 更轻量，WAR 包只有 11MB
"安全性怎么保证? "	BCrypt 12 rounds、RateLimiter 滑动窗口、前后端双校验正则、XSS 防护 escapeHtml、验证码 180s 冷却
"高并发怎么处理? "	HikariCP 连接池、游标分页（非 OFFSET）、WebSocket 多路复用、Canvas 粒子 requestAnimationFrame 管理
"AI 怎么集成的? "	OpenAI 兼容 API → DeepSeek、Transport 接口可测试、45s 超时容错、system prompt 定义 AI 人格
"最大的技术挑战是什么? "	WebRTC BusyUserRegistry → 内存 Set 改数据库查询、Schema 自动迁移避免手动 DDL、WebSocket 事件路由避免多连接

3.5 演示环节（控制 3 分钟内）

必须演示的 6 个点：

- 1. 用 admin 登录 → 左侧出现 ☹ 管理按钮 → 点进去看仪表盘 → 加分
- 2. 用 alice 和 bob 两个标签页 → 群聊中互发消息 → WebSocket 实时推送 → 核心
- 3. 写 @千问小助手 总结一下 → AI 回复 → wow 时刻
- 4. 发一条消息 → 截图发送粒子 + 弹簧动画 → 展示动效
- 5. 换群聊背景 → 全局玻璃质感变化 → 视觉冲击
- 6. 打开 <https://nbchatroom.ccloud> → 证明它真的在公网上运行

3.6 结语黄金 30 秒

"这个项目的代码全部开源在 GitHub 上，域名 nbchatroom.cloud 可以直接访问。我觉得这个项目最大的价值不在于它有多少功能，而在于它证明了一件事：**一个学生，用纯 Java 标准库和原生前端技术，不需要任何商业化框架，也能做出一个对标微信核心体验的聊天应用。** 谢谢各位老师。"

3.7 加分杀手锏

如果评委有时间，多说一句：

"如果给我一周时间，我最想加的功能是 AI 对话总结——当你错过了 200 条群消息，AI 帮你用三句话概括。这个功能在 Arattai 和 Telegram Premium 上已经有了，技术上我用 DeepSeek API 完全能做到。"

这表明：你不是在背诵功能列表，你是在思考产品问题。

四、补强优先级

优先级	动作	时间	答辩加分
●	AI 对话总结	1.5h	+30%
●	欢迎 Bot	1h	+20%
●	"跳转到最新"浮标	15min	+10%
●	快捷投票	1h	+20%
●	草稿保存	30min	+10%
●	流式 AI 回复	2h	+15%
●	最近表情	30min	+5%

报告生成时间：2026-06-14 · 基于 Arattai、XChat、Volla Messages、Signal、Stream Chat UX 等 12 个信息源综合